

## Stretch challenge answer key (instructor)

Instructor solutions for every stretch challenge in `stretch-tasks.pdf`. For each: the **expected observation**, **why it happens**, **what the trainee should conclude**, and **common mistakes**.

The numbers below were captured from the lab's **offline fallback backend** (the deterministic numpy hashing embedder). With all-MiniLM-L6-v2 the *absolute* scores differ, but the *trends* and conclusions are the same. Don't quote the exact decimals to trainees as if they were universal — the point is the shape of each result. This is itself the lesson of Stretch 4.

### Stretch 1 — Top-K experiment

**Expected observation.** For “Which flag did lumen sync add in version 2.3?”:

| top_k | Citations in the grounded answer                                   |
|-------|--------------------------------------------------------------------|
| 1     | 1 — lumen_cli.md > The sync command (0.415)                        |
| 3     | 2 — adds lumen_cli.md > Overview (0.189)                           |
| 5     | 2 — <b>unchanged</b> (extra retrieved chunks fall below the guard) |

Raising `top_k` from 3 to 5 pulls more chunks into *retrieval*, but the **answer does not change**: the new chunks (from `version_control.md` etc.) score  $\sim 0.15$  and are filtered out by the grounding guard.

**Why it happens.** `retrieve()` returns the top-K by similarity no matter what, but `generate_answer()` keeps only chunks at or above `MIN_RELEVANCE_SCORE`. Beyond the handful of genuinely relevant chunks, additional results are near-duplicates or noise. A bigger K just lengthens the candidate list; it doesn't manufacture new relevant evidence.

**What the trainee should conclude.** Top-K is a *recall vs. noise* dial, not a quality dial you can crank up for free. A small K risks missing a needed chunk; a large K dilutes the context with irrelevant text, costs tokens/latency, and gives the model more chances to anchor on the wrong passage. Pairing Top-K with a score threshold (the grounding guard) is what keeps the extra noise from reaching the answer.

**Common mistakes.** - Concluding “K=5 is better because it returned more sources” — count *useful* sources, not total ones. - Expecting the answer text to grow with K; here the guard holds it stable (a good thing). If a trainee's guard is disabled, the answer *will* fill with noise — a useful contrast to show. - Forgetting that with only 18-19 chunks in the toy corpus, K is capped by corpus size; the effect is more dramatic on a large corpus.

### Stretch 2 — Chunk size experiment

**Expected observation.** For “How do you define a function in Python?” (chunking the raw document text):

| size (words) | # chunks | top score    | words in top chunk |
|--------------|----------|--------------|--------------------|
| 15           | 72       | <b>0.472</b> | 15                 |
| 60           | 19       | 0.282        | 60                 |
| 400          | 5        | <b>0.179</b> | 190                |

A clean U-curve in disguise: **small** chunks score *highest* but each chunk is a tiny fragment; **large** chunks score *lowest* and the single retrieved chunk is  $\sim 190$  words of mostly-unrelated material (Return values, Parameters, Flexible arguments) wrapped around the one relevant sentence.

**Why it happens.** A chunk’s embedding is an average of its content. A 15-word chunk that is *only* about defining functions points almost exactly at the query — high cosine. A 400-word chunk mixes four topics, so its embedding is a blurry average that points *loosely* at the query — low cosine. Note the punchline: at size 400 the top score (0.179) falls **below the default guard (0.18)**, so the system would actually **refuse** — an over-large chunk diluted the signal until the right evidence looked irrelevant.

**What the trainee should conclude.** There is a sweet spot. Too small and a chunk is a context-free fragment (high score, low usefulness — a 15-word chunk may omit the qualifier you needed). Too large and the embedding is unfocused, retrieval degrades, and you also waste context tokens on irrelevant text. “A chunk should be about one thing” is the heuristic; the medium setting balances precision and context. The honest general answer is “it depends on the corpus and the embedding model” — hence tuning and evaluation.

**Common mistakes.** - Reading “small chunks have the highest score” as “small is best.” Score is not usefulness — a 15-word fragment can score well yet be too thin to answer from. - Not noticing the refusal at size 400; it’s the most instructive result — surface it. - Forgetting to restore the section-aware index (re-run Section 4) before continuing, then being confused by later cells. - Changing size but not overlap, then blaming size for a boundary-split sentence.

---

### Stretch 3 — Grounding guard (threshold)

**Expected observation.** Top scores: answerable (“404”) = **0.263**, unanswerable (“Mars”) = **0.000**.

| threshold | answerable refused?      | unanswerable refused? |
|-----------|--------------------------|-----------------------|
| 0.0       | No                       | <b>No</b> ← unsafe    |
| 0.1       | No                       | Yes                   |
| 0.2       | No                       | Yes                   |
| 0.4       | <b>Yes</b> ← over-strict | Yes                   |
| 0.8       | Yes                      | Yes                   |

**Why it happens.** The guard refuses when the top score is *strictly below* the threshold. At 0.0, Mars (score 0.0) is **not** below 0.0, so it slips through and is “answered” from zero-relevance chunks — the classic unsafe default. Thresholds in roughly  $(0.0, 0.26]$  answer the 404 question (top 0.263) while refusing Mars (top 0.0). Push the threshold above 0.263 and even the legitimate question is refused.

**What the trainee should conclude.** The grounding guard is the knob that converts “retrieval always returns something” into “answer only when the something is relevant enough.” Setting it is a precision/recall trade-off: too low → answers unanswerable questions (false confidence); too high → refuses answerable ones (over-caution). In a high-stakes domain, the over-caution failure is the *safer* one — refusing a real question is recoverable; confidently answering a question you have no evidence for is not. There is no universal number: it depends on the corpus and, critically, the embedding model’s score scale.

**Common mistakes.** - Using 0.0 “to be safe” — it’s the opposite of safe; it never refuses anything that retrieved any chunk at all. - Treating the threshold as  $\leq$  vs  $<$ ; the boundary case (Mars at exactly 0.0) is why the comparison operator matters. Point it out. - Hard-coding a threshold found here and reusing it with MiniLM or another corpus (see Stretch 4) — the scales don’t transfer. - Concluding a single question proves a threshold; a real guard is calibrated on a labelled set (this is the production system’s evaluation work, out of scope here).

---

## Stretch 4 (optional, advanced) — Embeddings bake-off

**Expected observation.** The query “How do I **declare** a **procedure** in Python?” shares almost no words with the source sentence (“A function in Python is **defined** using the **def** keyword...”):

- **Offline fallback (bag-of-words):** top score **0.084**, retrieving the **wrong** chunk (`version_control.md > Commits`). Below the 0.18 guard → it would **refuse** (or answer from an irrelevant chunk). The keyword overlap is nearly zero, so it fails.
- **all-MiniLM-L6-v2 (expected, run online):** retrieves the correct `python_functions.md > Defining a function` chunk with a healthy score, because the model maps “declare a procedure” and “define a function” to nearby points in meaning space despite the different words.

(For the other two queries, both backends agree: Lumen ~0.415 fallback / high for MiniLM, and Mars ~0.0 → refuse under both.)

**Why it happens.** The fallback is a *lexical* embedder — it can only match shared tokens, so synonyms and paraphrases are invisible to it. A trained sentence embedder is *semantic* — it learned from data that “declare a procedure” and “define a function” mean nearly the same thing. This “vocabulary gap” is precisely where semantic search beats keyword search.

**What the trainee should conclude.** Embedding **quality** directly sets retrieval quality — the rest of the pipeline can’t recover a fact the retriever couldn’t find (tie back to Slide 9: inspect retrieval first). It also shows that raw score numbers are **backend-specific**: 0.4 means different things under the two embedders, so a threshold tuned for one won’t transfer to the other (ties back to Stretch 3). This experiment is the concrete form of **Misconception 3** (“vector search always retrieves the correct answer”) — a weak embedder fails the vocabulary gap outright.

**Common mistakes.** - Comparing the two backends’ *absolute* scores and concluding one is “more confident.” They’re different scales; compare *ranking correctness*, not the decimals. - Assuming the fallback is “broken.” It works for keyword-overlapping queries; it simply can’t bridge synonyms — which is exactly the point. - Forgetting that MiniLM also has limits: it will still miss truly out-of-distribution or highly domain-specific nuance. Better is not perfect (Misconception 4).